



PDF Download
3779212.3790223.pdf
23 March 2026
Total Citations: 0
Total Downloads: 0

Latest updates: <https://dl.acm.org/doi/10.1145/3779212.3790223>

RESEARCH-ARTICLE

SNIP: An Adaptive Mixed Precision Framework for Subbyte Large Language Model Training

YUNJIE PAN, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

YONGYI YANG, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

HANMEI YANG, University of Massachusetts Amherst, Amherst, MA, United States

SCOTT MAHLKE, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

Open Access Support provided by:

University of Massachusetts Amherst

University of Michigan, Ann Arbor

Published: 22 March 2026

Citation in BibTeX format

ASPLOS '26: 31st ACM International
Conference on Architectural Support for
Programming Languages and Operating
Systems

March 22 - 26, 2026
PA, Pittsburgh, USA

Conference Sponsors:

SIGOPS
SIGPLAN
SIGARCH

SNIP: An Adaptive Mixed Precision Framework for Subbyte Large Language Model Training

Yunjie Pan
University of Michigan
Ann Arbor, Michigan, USA
panyj@umich.edu

Hanmei Yang
University of Massachusetts Amherst
Amherst, Massachusetts, USA
hanmeiyang@umass.edu

Yongyi Yang
University of Michigan
Ann Arbor, Michigan, USA
NTT Research, Inc.
Sunnyvale, California, USA
yongyi@umich.edu

Scott Mahlke
University of Michigan
Ann Arbor, Michigan, USA
mahlke@umich.edu

Abstract

Training large language models (LLMs) efficiently while preserving model quality poses significant challenges, particularly with subbyte precision supported by state-of-the-art GPUs. Current mixed-precision training approaches either apply uniform precision to all GEMM operations or rely on heuristic-based methods that fail to generalize during training, leading to suboptimal convergence and instability.

To address these challenges, this paper introduces SNIP, a fine-grained adaptive mixed-precision training framework for LLM pretraining that supports subbyte precision. SNIP periodically collects statistics on activations, gradients, and optimizer states to assess the precision loss impact on model quality. We define two key metrics: *loss divergence* in the forward pass, caused by quantization-induced increases in training loss, and *weight divergence* in the backward pass, which measures error propagation through gradients affecting model updates. These metrics guide an Integer Linear Programming (ILP) problem that systematically optimizes layerwise precision to minimize overall quality loss while meeting efficiency targets. Experiments on 1B, 3B, 7B and 70B Llama-like models demonstrate that SNIP consistently outperforms existing baselines, reducing FLOPs by up to 80% while preserving model quality across different model sizes and training phases with minimal computational overhead.

CCS Concepts: • Computer systems organization → Architectures; • Computing methodologies → Neural networks.

Keywords: Mixed-precision training, Quantization, Large language models, Neural network

ACM Reference Format:

Yunjie Pan, Yongyi Yang, Hanmei Yang, and Scott Mahlke. 2026. SNIP: An Adaptive Mixed Precision Framework for Subbyte Large Language Model Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3779212.3790223>

1 Introduction

Large Language Models (LLMs) have revolutionized natural language processing (NLP), powering applications such as conversational AI, code generation, and reasoning [2, 5, 8, 22, 44, 66, 67]. Despite their widespread utility, training LLMs demands extraordinary computational resources. For example, training Llama 3 [22] with 8 billion parameters requires 1.46 million GPU hours on NVIDIA H100 GPUs, resulting in 420 tons of CO2 emissions. The immense computational resources, runtime, and environmental impact underscore the urgent need for efficient training strategies.

Mixed precision training [49] has emerged as a pivotal technique to mitigate the costs, allowing most compute-intensive operations to be executed in low-precision formats (e.g. FP8 [44, 50, 57], FP4 [73]), while retaining higher precision for critical operations. The NVIDIA Hopper GPU supports the FP8 format [54], while the more recent Blackwell GPU extends support to the FP4 format [55]. For the FP8 format, the GEMM operation achieves twice the TFLOPS of BF16, resulting in a 1.3x to 1.4x speedup in end-to-end LLM training. Similarly, an FP4 GEMM on the Blackwell GPU doubles the TFLOPS compared to FP8, which translates into an additional 1.4x speedup over FP8 training latency [68].

However, most existing mixed precision training frameworks adopt a uniform precision policy, assuming identical precision requirements for all layers, thereby missing



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790223>

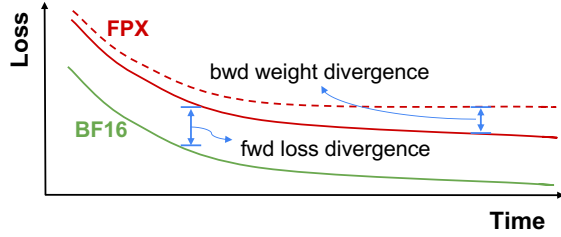


Figure 1. Illustration of the gap of the training loss between high-precision (BF16) and low-precision (FPX). The gap consists of two parts: (1) Forward Loss Divergence: the increase in training loss directly introduced by quantization during the forward pass, and (2) Backward Weight Divergence: the accumulation of errors in weight updates during the backward pass due to quantization, which can compound over iterations and layers, impacting model convergence.

the opportunity for greater efficiency gains through fine-grained precision configurations [44, 50, 57, 73]. While prior research has investigated layer-wise quantization for machine learning models, these efforts are largely focused on inference rather than training and come with limitations. These methods often rely on local quantization metrics such as KL-divergence [12, 72], absolute quantization error, or relative quantization error, which fail to fully capture the broader training dynamics. Additionally, some prior work employs computationally expensive approaches, including reinforcement learning (HAQ [72]), Hessian-based sensitivity analysis (OBC [24]), or iterative precision refinement (BitSET [56]). These techniques are primarily tailored for inference and do not address the comprehensive needs of model quality during LLM pretraining, thus limiting their effectiveness in a training context.

To address these limitations, we propose SNIP, a fine-grained mixed precision training framework that dynamically determines the quantization per layer to ensure training quality and efficiency. Rather than relying on exhaustive search or purely heuristic-based decisions, SNIP introduces a novel optimization proxy quality metric to quantify the impact of quantization.

SNIP quantifies training quality loss through two key metrics, as illustrated in Figure 1: loss divergence and weight divergence. Loss divergence occurs in the forward pass when quantization increases training loss. Weight divergence occurs in the backward pass when quantization errors in gradients distort parameter updates, resulting in the trained model deviating from the models trained with full precision. Building upon these metrics, we formulate the problem of determining layer-specific quantization schemes as an integer linear programming (ILP) problem. The ILP systematically minimizes quality loss while meeting efficiency constraints, ensuring a globally optimized quantization policy.

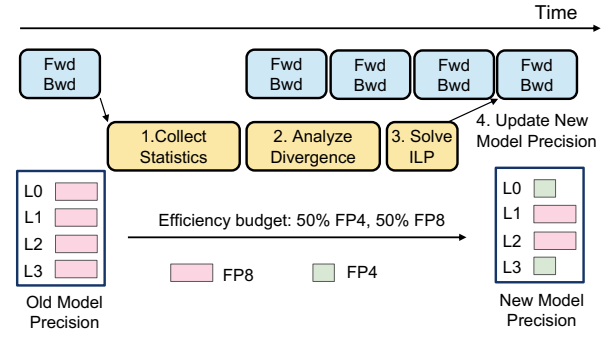


Figure 2. System overview of SNIP, showing its integration into LLM training. SNIP periodically collects statistics on activations, weights, and optimizers, then asynchronously analyzes divergence metrics, solves an ILP problem, and updates layer-wise quantization.

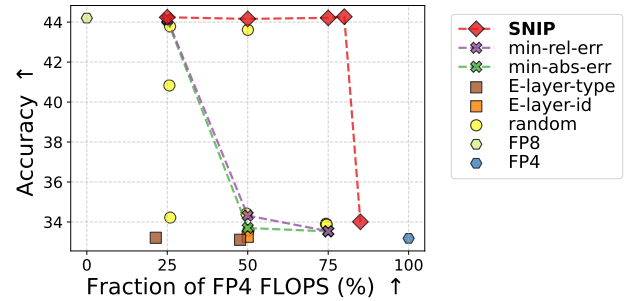


Figure 3. Comparison of accuracy versus efficiency (the fraction of FP4 FLOPs) for the TinyLlama 1B model. The FP8 baseline achieves the highest accuracy but lowest efficiency, while the FP4 configuration maximizes efficiency at the cost of accuracy. SNIP demonstrates a balance between high accuracy and a significant reduction in FLOPs by selectively applying FP4 to certain layers based on the loss and weight divergence metrics, outperforming other methods such as *random* and other heuristic-based approaches.

Beyond its theoretical advancements, SNIP designs a practical system that integrates seamlessly into LLM training pipelines, as shown in Figure 2. The system operates asynchronously alongside standard training, periodically updating layer-wise quantization decisions without interrupting the primary training loop. The system follows a structured workflow: (1) collect statistics on activations, gradients, and optimizer states, (2) analyze these metrics to estimate the impact of precision scaling, (3) formulate an optimization problem to determine the optimal per-layer precision settings, and (4) update quantization assignments asynchronously. This adaptive approach ensures that SNIP remains effective across different training phases and model architectures, making it broadly applicable to various LLM workloads.

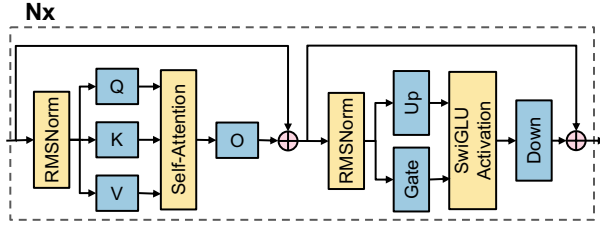


Figure 4. The transformer block structure of Llama-like LLM. The blocks in blue (Q, K, V, O, Gate, Up and Down) are linear layers.

Figure 3 highlights the trade-off between the accuracy and efficiency of different precision selection methods for the TinyLlama 1B model. Efficiency is measured as the fraction of FLOPs executed in FP4, with the remainder using FP8 precision. SNIP consistently outperforms all other quantization methods at different efficiency levels, including *random* (randomly selecting layers for FP4), *min-abs-err* (selecting layers that minimize absolute quantization error for FP4), *min-rel-err* (selecting layers that minimize relative quantization error for FP4), *E-layer-type* (empirically apply FP4 to non-sensitive layer types), and *E-layer-id* (empirically apply FP4 for middle layers). Remarkably, even at 80% FP4 FLOPs, SNIP maintains nearly full-precision accuracy, whereas alternative quantization methods fail to converge.

The contributions of this paper are:

- We propose SNIP a fine-grained mixed-precision quantization framework specifically for LLM pretraining that seamlessly integrates into existing pipelines. SNIP periodically determines the optimal quantization policy while minimizing the model quality loss. Unlike heuristic-based methods, SNIP provides a global optimization strategy that dynamically adapts to different training stages and supports various quantization techniques.
- We present a novel theoretical perspective on how quantization errors influence overall LLM training quality. Specifically, we introduce two quantization quality metrics, loss divergence for the forward pass and weight divergence for the backward pass, which quantify the impact of precision scaling on training stability. These metrics enable efficient precision selection with minimal overhead.
- We evaluate SNIP on 1B, 3B, 7B, and 70B models using up to 80% FP4 FLOPs, demonstrating that it consistently improves training efficiency with subbyte precision while maintaining near full-precision model accuracy.

2 Background

2.1 LLM Structure

Most LLMs are built upon the Transformer [69] framework, which consists of embedding layers that convert tokenized inputs into dense vector representations, transformer blocks

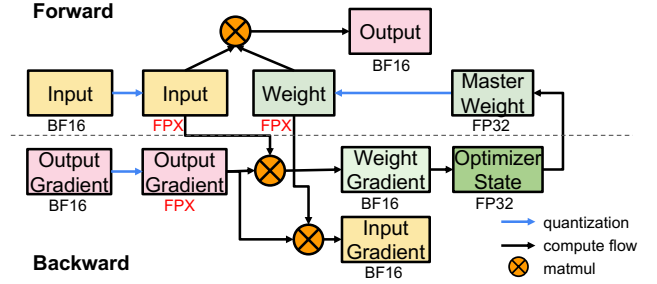


Figure 5. The overall mixed precision training framework for linear layers, including the forward and backward pass.

that are the core computation units, stacked N times to increase the model capacity, and output projection layer that maps the final hidden representations of tokens back to the vocabulary space. We illustrate the transformer block structure of Llama-like LLMs in Figure 4. Each transformer block contains multi-head self-attention, which uses three types of linear layers Query (Q), Key (K), and Value (V) to project inputs to an intermediate representation. After computing the attention scores, the output is projected back to the original dimension using the Output linear layer (O). Subsequently, the processed tensors are passed through a Feedforward Neural Network (FFN), which comprises the Gate and Up linear layers for intermediate transformations. After the non-linear activation (e.g. SwiGLU), the intermediate representation is projected back by the Down linear layer. Following prior works [44, 73], we focus on the quantization of those linear layers in transformer blocks because they take the majority (>90%) of the FLOPs during training [9].

2.2 Mixed Precision Training

Mixed precision training [49] addresses training costs and model accuracy by using reduced precision, typically BF16, in linear or convolutional layers while retaining higher precision for critical operations. State-of-the-art advancements such as DeepSeek-V3 [44] have extended these techniques to include FP8 training for large language models (LLMs). Meanwhile, Wang *et al.* [73] have integrated FP4 into mixed precision training. The FP4 approaches, while employing FP4 across all layers, necessitate complex gradient calculations that add overhead, and rely on irregular sparse GEMM to handle outliers, which further adds significant overhead.

Adopting a similar idea on mixed precision training for LLMs, we present the overall framework used in our experiments in Figure 5. In this framework, the most compute-intensive operators (GEMM) within linear layers are executed in low precision (FPX), where FPX refers to low-precision formats such as FP8 or FP4. Before the GEMM operation, the input activations, weights, and output gradients are quantized to low precision, while the output of the GEMM operator remains in BF16. To ensure numerical

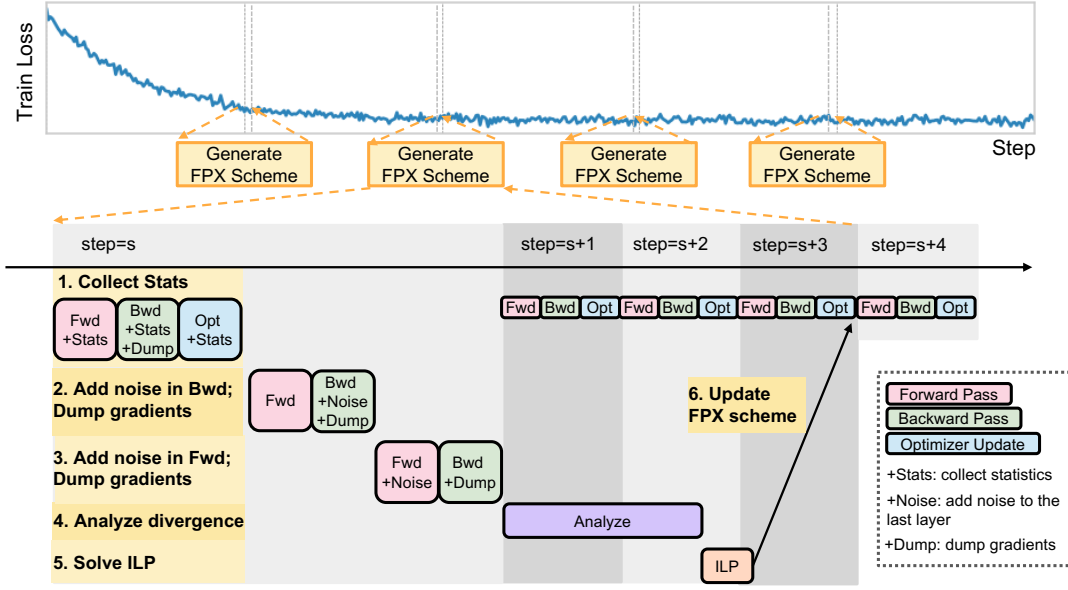


Figure 6. An overview of the SNIP training workflow. The top plot illustrates the training loss over steps, with periodic updates to the FPX quantization scheme highlighted in yellow boxes. The process consists of six steps: (1) collecting statistics during a normal iteration, (2) injecting noise during the backward pass and dumping gradients, (3) injecting noise during the forward pass and dumping gradients, (4) analyzing loss and weight divergence, (5) solving the Integer Linear Programming (ILP) problem to determine the optimal FPX quantization scheme, and (6) updating the FPX scheme.

stability, we maintain a master copy of the weights in FP32, following DeepSeek-V3 [44]. Other types of layers, such as RMSNorm, SwiGLU, Softmax, and Attention, continue to use higher precision (BF16) to preserve accuracy while optimizing overall efficiency. This hybrid approach ensures a balanced trade-off between computational cost and model performance.

Mixed precision training with low-precision formats such as FP4 and FP8 reduces both computation and communication overhead. On NVIDIA Blackwell [55], FP4 offers 2× the throughput of FP8 and 4× that of BF16. Besides, storing weights in FP4/FP8 also reduces HBM storage cost, which is the main bottleneck in large-scale LLM training. For communication, pipeline parallelism ensures that FSDP communication is not on the critical path for the training. Extending low-precision support to reduce-scatter is a promising but challenging direction for future work.

2.3 Quantization

Quantization Format. Common floating point types for training include IEEE single precision (FP32) and bfloat16 (BF16) [36]. To reduce the number of bits in floating point further, FP8 format has been proposed and integrated into commercial GPUs. There are different FP8 formats studied [50, 63]: E5M2, E4M3 and E3M4, where the names indicate the number of exponent (E) and mantissa (M) bits, respectively. Following MX specification [60], we adopt the FP4 E2M1 format.

Quantization granularity. Because low-precision formats (e.g., FP8 and FP4) have a limited dynamic range, scaling factors are essential to mitigate numerical overflows and underflows. This is particularly crucial during training, as gradients often exhibit a larger dynamic range [44], and their numerical accuracy significantly affects model convergence. As a standard practice, the input distribution is aligned to the representable range of the FPX format by scaling the maximum absolute value of the input tensor to the maximum representable value of FPX (FPX_MAX).

$$scale = FPX_MAX / \max(abs(x))$$

Before applying the low-precision operator (Op), the inputs are scaled by the scaling factor and then quantized to the low-precision format. After the operator, the output is scaled back by dividing by the same scaling factor.

$$y = Op(Quant(x * scale)) / scale$$

To reduce quantization error, tensors can be scaled at different levels of granularity: (1) tensorwise: Every tensor has a scaling factor; (2) rowwise/columnwise: Every row or column of a tensor has a scaling factor; (3) blockwise: Every $N_B \times N_B$ block of a tensor has a scaling factor; (4) tilewise: Every $1 \times N_B$ tile of a tensor has a scaling factor. Following DeepSeek-V3 [44]’s strategy, we apply 1×128 tilewise quantization for input activations and gradients, and 128×128 blockwise quantization for weights.

3 Overall Procedure

In this section, we present the overall process for generating layer-wise quantization schemes (FPX schemes) during the pretraining of LLMs, as illustrated in Figure 6. Throughout the training process, optimal quantization schemes are periodically generated and applied asynchronously, ensuring seamless integration with the normal training workflow.

Specifically, the generation of FPX schemes consists of two main tasks: (a) Collecting Statistics and Analyzing Divergence; (b) Determining the optimal layer-wise quantization scheme. Task (a) comprises Steps 1 through 4, while Task (b) includes Steps 5 and 6. We use the example of Figure 6 that collects the statistics regarding the checkpoint at step s to generate the optimal quantization scheme.

3.1 Collecting Statistics and Analyzing Divergence

In Step 1 of Figure 6, we collect statistics during a standard training iteration using high precision (BF16), which includes the forward pass, backward pass, and optimizer updates, while also dumping gradient tensors. Specifically, during the forward and backward passes, we record key statistics such as the Frobenius norm of inputs, weights, outputs, output gradients, input gradients, and weight gradients. Additionally, we compute and store the Frobenius norm of the quantization error for these components. During the optimizer update, we gather statistics like the first and second moments to accurately model weight update dynamics. Since most statistics involve simple Frobenius norm computations, the overhead of collecting statistics is negligible compared to a standard training iteration. The theoretical justification for collecting this information is discussed in detail in Section 4.

Steps 2 and 3 approximate the Frobenius norm of the second-order derivative to estimate the impact of quantization on the backward weights update, as calculating its exact value is computationally prohibitive. The theoretical rationale behind these steps is discussed in detail in Section 4.3. In Step 2, using the same batch of data from training step s , we inject small Gaussian noise into the last layer during the backward pass and save the gradient tensors. Later, these gradients are compared with the baseline gradients (collected in Step 1 without noise) by computing the Frobenius norm of their difference. Similarly, in Step 3, Gaussian noise is added to the last layer during the forward pass, and the resulting gradients are compared with the baseline gradients. Step 2 and 3 require two additional forward and backward passes without updating the weights that is done on GPU, so these 2 steps introduce some computational overhead.

In Step 4, we leverage the collected statistics and dumped tensors to analyze and compute two key metrics: loss divergence, which captures the increase in training loss caused by quantization errors during the forward pass, and weight divergence, which quantifies the deviation of the weights from the baseline model due to quantization errors. This analysis

can be offloaded to the CPU, allowing the normal training process to continue seamlessly with subsequent steps, as illustrated in the example for steps $s + 1$ and $s + 2$ in Figure 6.

Our algorithm is agnostic to the parallelism strategy and requires only minor implementation considerations. The 70B experiments in Section 6 confirm that it works seamlessly under different parallelism configurations.

3.2 Deciding the Optimal Layer-wise Quantization Scheme

In Step 5, the loss and weight divergence are used to formulate an Integer Linear Programming (ILP) problem. The goal of this optimization is to identify the optimal quantization scheme for each layer, balancing training quality and efficiency requirements. The ILP maps the problem to a knapsack model, where each layer is a decision variable, and precision formats (e.g., FP8, FP4) are quantization options. SNIP is compatible with emerging quantization techniques, as new methods can be incorporated as additional quantization options. The objective function minimizes a weighted combination of the total loss divergence and weight divergence across all layers, subject to efficiency constraints (e.g., the fraction of FLOPs allocated to FP4 precision).

After solving the ILP problem in Step 5, the optimal quantization scheme is applied to the training process in Step 6. To minimize overhead and maintain training efficiency, the updated scheme is applied asynchronously without interrupting the ongoing training iterations. The updated scheme remains in effect until the next quantization update cycle.

4 Quantify the Quantization Impact

4.1 Preliminary

Figure 5 shows the quantization process in LLM mixed precision training. We only consider linear layers in transformer blocks, whose computation is matrix multiplication.

Quantization reduces the number of bits used to present the tensors of inputs, weights, and gradients in DNNs, that incurs error. Following prior work [6, 39], we model the impact of quantization of a tensor as a small random perturbation term. Specifically,

$$q(x) = x + \delta_x$$

where x is the ground truth value of a tensor, $q(x)$ is the quantized value of x , and δ_x is a small random Gaussian vector.

We denote $\|\cdot\|_F$ as the Frobenius norm for a matrix (equivalent to the ℓ_2 norm of the vectorization of a matrix) and the ℓ_2 norm for a vector. We use I_d to represent the identity matrix of size $d \times d$, and $\mathcal{N}$ to represent the Gaussian distribution. Below we prove two theorems that are critical to our analysis. The idea behind these theorems is that if the perturbation is random enough, then the error can be estimated much more accurately than a trivial one (say, using

the Lipschitz constant). Similar ideas have been adopted in recent work on quantization under different settings [26, 80].

Theorem 4.1. *Let $\epsilon > 0$ be a small scalar, $\delta_x \sim \mathcal{N}\left(0, \frac{\epsilon^2}{d} I_d\right)$ be a random Gaussian vector, and $g : \mathbb{R}^d \rightarrow \mathbb{R}^m$ be a smooth function with S -Lipschitz gradient such that $S \ll \frac{1}{\epsilon^2}$, then the following inequality holds with probability at least 0.99:*

$$\|g(x + \delta_x) - g(x)\|_F \lesssim \|\nabla g(x)\|_F \frac{\epsilon}{\sqrt{d}},$$

where we use $\lesssim$ to hide constant coefficients.

Theorem 4.1 illustrates how we can use the gradient norm to estimate the error under a small perturbation. Conversely, we can also use the error under perturbation to estimate the gradient norm.

Theorem 4.2. *Let $g : \mathbb{R}^d \rightarrow \mathbb{R}^m$ be a smooth function, and let $\delta_x \sim \mathcal{N}(0, \epsilon^2 I_d)$ be a standard Gaussian vector, then we have*

$$\|\nabla_x g(x)\|_F^2 = \lim_{\epsilon \rightarrow 0} \epsilon^{-2} \mathbb{E} \|g(x) - g(x + \delta_x)\|_F^2.$$

4.2 Loss Divergence in Forward Pass

During the forward pass, quantizing activations and weights introduces perturbations that propagate through subsequent layers, ultimately affecting the loss. For a given layer l , the loss can be represented as a $L(X_l, W_l)$, where $X_l \in \mathbb{R}^{M_l \times K_l}$ and $W_l \in \mathbb{R}^{N_l \times K_l}$ are the activations and weights of layer l . Using Theorem 4.1, we estimate the impact of quantizing a tensor by the following approximations:

$$\begin{aligned} & \|L(X_l + \delta_{X_l}, W_l + \delta_{W_l}) - L(X_l, W_l)\| \\ & \approx \sqrt{\|L(X_l + \delta_{X_l}, W_l) - L(X_l, W_l)\|^2 + \|L(X_l, W_l + \delta_{W_l}) - L(X_l, W_l)\|^2}, \end{aligned}$$

where

$$\|L(X_l + \delta_{X_l}, W_l) - L(X_l, W_l)\|_F \approx \frac{\|\nabla_{X_l} L\|_F \|\delta_{X_l}\|_F}{\sqrt{M_l K_l}}$$

and

$$\|L(X_l, W_l + \delta_{W_l}) - L(X_l, W_l)\|_F \approx \frac{\|\nabla_{W_l} L\|_F \|\delta_{W_l}\|_F}{\sqrt{N_l K_l}}.$$

Notice that we omit the cross term $L(X_l + \delta_{X_l}, W_l + \delta_{W_l}) - L(X_l, W_l)$ here, since it is of magnitude $O(\|\delta_{X_l}\| \|\delta_{W_l}\|)$ which is small compared to the other terms.

Here, $\nabla_{X_l} L$ and $\nabla_{W_l} L$ are the gradient of L with respect to activation X_l and weights W_l , respectively. These gradients are naturally computed during the backward pass of LLM training, allowing us to quantify the impact of quantization on loss with minimal computational overhead in Step 1 of Figure 6.

We define the normalized loss divergence as follows.

Definition 4.3. Normalized loss divergence $\Delta L = |L' - L|/|L|$, where L and L' are the loss of the model during forward pass without and with quantization, respectively.

4.3 Weight Divergence in Backward Pass

4.3.1 Impact on Weight Gradient. In the backward pass, the quantization error of a given layer affects the weight gradient of that layer as well as all preceding layers. Consider quantizing the input activation $X_l \in \mathbb{R}^{M_l \times K_l}$, weights $W_l \in \mathbb{R}^{N_l \times K_l}$, and output gradient $\nabla_{Y_l} L \in \mathbb{R}^{M_l \times N_l}$ of layer l . The weight gradient at layer l , denoted as $\nabla_{W_l} L$, can be represented as $g_l(X_j, W_j, \nabla_{Y_j} L)$, for any $j \geq l$. This implies that the weight gradient at layer l is influenced not only by the quantization at the current layer but also by quantization errors in all subsequent layers. We estimate the error by the following:

$$\|g_l(X_j + \delta_{X_j}, W_j, \nabla_{Y_j} L) - g_l(X_j, W_j, \nabla_{Y_j} L)\|_F \approx \frac{\|\nabla_{X_j} g_l\|_F \|\delta_{X_j}\|_F}{\sqrt{M_j K_j}},$$

and

$$\|g_l(X_j, W_j + \delta_{W_j}, \nabla_{Y_j} L) - g_l(X_j, W_j, \nabla_{Y_j} L)\|_F \approx \frac{\|\nabla_{W_j} g_l\|_F \|\delta_{W_j}\|_F}{\sqrt{N_j K_j}}.$$

Note that $\|\nabla_{X_j} g_l\|_F = \left\| \frac{\partial(\frac{\partial L}{\partial W_l})}{\partial X_j} \right\|_F$ and $\|\nabla_{W_j} g_l\|_F = \left\| \frac{\partial(\frac{\partial L}{\partial W_l})}{\partial W_j} \right\|_F$ are second-order partial derivatives, which is not computed during the forward or backward pass in LLM training. To estimate $\|\nabla_{X_j} g_l\|_F$ and $\|\nabla_{W_j} g_l\|_F$, we apply Theorem 4.2. Here, we approximate the expectation by a single sample per batch and the limit by taking a very small value. This estimation requires additional backward passes with added noise, shown as Step 2 and 3 in Figure 6.

4.3.2 Impact on Weight Divergence. In the optimizer update step, the quantization error of gradients $g_l = \nabla_{W_l} L$ propagates into the model parameters W_l . We formally define the normalized weight divergence as follows.

Definition 4.4. $\Delta W = \sum_l \frac{1}{N} (\|W'_l - W_l\|_F / \|W_l\|_F)$, where W_l and W'_l represent the weights at layer l after the optimizer updates the model parameters without and with quantization, respectively. And N is the number of layers.

For this analysis, we focus on the AdamW optimizer [46], the most widely adopted optimizer in LLM training [5, 22, 44, 67]. Without loss of generality, our method applies to any differentiable optimizer. For simplification, we focus on a specific layer l , omitting the layer subscript l in subsequent expressions. The AdamW optimizer updates parameters at each step t as follows:

$$\begin{aligned}
W_{t-1} &\leftarrow W_{t-1} - \alpha \lambda W_{t-1} \\
m_t &\leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \\
v_t &\leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\
\hat{m}_t &\leftarrow \frac{m_t}{1 - \beta_1^t} \\
\hat{v}_t &\leftarrow \frac{v_t}{1 - \beta_2^t} \\
W_t &\leftarrow W_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}
\end{aligned}$$

where α is the learning rate, β_1, β_2 are exponential decay rates, λ is weight decay, and $\epsilon > 0$ is a small constant for numeric stability, and g_t is the gradients of weights at step t .

Let us define W_t and W'_t as the weights (model parameters) without and with quantization in the backward pass, respectively.

$$W_t - W'_t = (1 - \alpha\lambda)(W_{t-1} - W'_{t-1}) + \alpha \left(\frac{\hat{m}'_t}{\sqrt{\hat{v}'_t} + \epsilon} - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right)$$

The first term $(1 - \alpha\lambda)(W_{t-1} - W'_{t-1})$ represents the accumulated deviation from previous iterations due to quantization, which compounds over time. So we focus only on the second term, which captures the quantization effect at the current iteration t .

$$\begin{aligned}
&\alpha \left(\frac{\hat{m}'_t}{\sqrt{\hat{v}'_t} + \epsilon} - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \\
&= \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t} \left(\frac{m'_t}{\sqrt{v'_t} + \epsilon} - \frac{m_t}{\sqrt{v_t} + \epsilon} \right)
\end{aligned}$$

Within the same iteration, the values of α, β_1, β_2 , and δ remain consistent across all layers. Notably, the norms $\|m'_t - m_t\|_F$ and $\|v'_t - v_t\|_F$ are much smaller than $\|g'_t - g_t\|_F$, as $(1 - \beta_1)$ and $(1 - \beta_2)$ are typically very small (often < 0.1). Therefore, we consider the following function of gradient g to represent the weight divergence due to quantization:

$$h(g) = \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t} \frac{m_t}{\sqrt{v_t} + \epsilon}$$

For simplicity, we focus only on iteration t and omit the subscript t from g_t .

Using the Theorem 4.1, we have that

$$\begin{aligned}
&\|h(g + \epsilon_g) - h(g)\|_F \\
&\approx \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t} \left\| \frac{1 - \beta_1}{\sqrt{v_t} + \epsilon} - \frac{(1 - \beta_2)m_t g_t}{\sqrt{v_t}(\sqrt{v_t} + \epsilon)^2} \right\|_F \frac{\|\epsilon_g\|_F}{\sqrt{NK}}
\end{aligned}$$

For implementation, in Step 1 of Figure 6, we collect statistics during a standard training iteration using the AdamW optimizer. These include β_1, β_2 , and the Frobenius norm of the term $\frac{1 - \beta_1}{\sqrt{v_t} + \epsilon} - \frac{(1 - \beta_2)m_t g_t}{\sqrt{v_t}(\sqrt{v_t} + \epsilon)^2}$.

5 Find Optimal Quantization Policy

Given the estimated impact on quality loss and efficiency improvement, we now introduce the method to determine the optimal quantization policy that satisfies efficiency constraints while minimizing quality loss.

5.1 Quality Loss and Efficiency Metrics

To formulate the problem, we quantitatively define the metrics for quality loss Q and efficiency E . For quality loss Q , we used a weighted sum of the normalized loss divergence ΔL (defined in Definition 4.3) in the forward pass and the normalized weight divergence ΔW (defined in Definition 4.4) in the backward pass. Then $Q = \Delta L + \Delta W$.

For efficiency E , since we do not have access to GPUs that natively support both FP8 and FP4 formats, we cannot directly measure the end-to-end performance improvement of low-precision formats compared to full precision (BF16). Therefore, we approximate E as the fraction of FLOPs executed in FP4 format, where the remaining $1 - E$ fraction of FLOPs is executed using the FP8 format. This provides a practical proxy for efficiency in our evaluation.

5.2 Integer Linear Programming Problem

The problem of finding the optimal quantization settings that meet the efficiency improvement requirements while minimizing the training quality can be mapped to a modified knapsack problem. Therefore, we could solve the problem by a Integer Linear Programming Problem (ILP), which ensures globally optimal solutions under the given constraints.

In our case, the *value* corresponds to the negative of training quality loss Q , and the *weight* corresponds to the negative of efficiency improvement E . Each layer of the model is viewed as an "item", with multiple possible quantization settings from which only one can be selected. For each layer i , the options are combinations of FP8 and FP4 formats for inputs, weights, and gradients. With m layers and n options per layer, our objective is to minimize overall quality loss while achieving the targeted efficiency improvement E_t .

We define the following symbols for clarity:

- $q_{i,j}$ is the quality loss of selecting the j -th option for layer i .
- $e_{i,j}$ is the efficiency saving of selecting the j -th option for layer i .
- $x_{i,j} \in \{0, 1\}$ is a binary decision variable indicating if the j -th option for layer i is selected.
- E_t is a floating point number in $[0, 1]$ that represents the target efficiency improvement. For example, $E_t = 0.5$ indicates that half of the FLOPs are executed in FP4.

The ILP formulation guarantees an optimal solution exists for any E_t in $[0, 1]$. When $E_t = 0$, the optimal solution assigns FP8 precision to all linear layers. And when $E_t = 1$, the optimal solution assigns FP4 precision to all linear layers.

$$\{x_{i,j}^*\} = \arg \min_{\{x_{i,j}\}} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} q_{i,j} x_{i,j} \quad (1)$$

$$\text{s.t.} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} e_{i,j} x_{i,j} \geq E_t \quad (2)$$

$$\sum_{j=0}^{n-1} x_{i,j} = 1, \quad \forall i \in \{0, 1, \dots, m-1\} \quad (3)$$

$$x_{i,j} \in \{0, 1\}, \quad \forall i, j \quad (4)$$

5.3 Incorporating Pipeline Parallelism

Pipeline parallelism [52] is widely used in LLM training to enhance memory and compute efficiency by partitioning the model's layers into stages that process data concurrently. However, imbalances in computation time across pipeline stages can create bubbles, which bottleneck efficiency improvements at the slowest stage. To address this, we extend the ILP problem to explicitly consider pipeline parallelism by ensuring balanced efficiency savings across all stages. We model this as a grouped knapsack problem, where each group represents a pipeline stage, and the objective is to select an optimal quantization scheme per layer while maintaining balanced efficiency across stages.

We set some additional definitions. K is the number of groups, and g is the number of items in every group. Formally, we modify the prior ILP formulation by replacing the efficiency constraint line (2) with the following group-aware (pipeline-stage-aware) constraint:

$$\text{s.t.} \sum_{i=k*g}^{(k+1)*g-1} \sum_{j=0}^{n-1} e_{i,j} x_{i,j} \geq \frac{E_t}{K}, \forall k \in \{0, 1, \dots, K\} \quad (5)$$

This ensures that each pipeline stage contributes equally to the overall efficiency target, preventing bottlenecks and maintaining a well-balanced workload across stages.

6 Evaluation

6.1 Experiment Setup

Models. We conduct our pretraining experiments using open-source models and datasets, leveraging publicly available intermediate checkpoints to ensure reproducibility and accessibility. Specifically, we use the TinyLlama 1B [84] and OpenLlama 3B, OpenLlama 7B [28] models, as well as an industry LLaMA-style dense 70B model. Given that training TinyLlama 1B from scratch on 300B tokens requires approximately 3,456 A100 GPU hours [84], we opt to resume pretraining from the released intermediate checkpoints for open source models.

Specifically, for the TinyLlama 1B model, we use checkpoints at 5K steps (10B tokens), 10K steps (21B tokens), 20K steps (42B tokens), 50K steps (105B tokens), and 240K steps

(503B tokens). For OpenLlama 3B and 7B models, we use checkpoints at 50k steps and 100k steps. Since the released checkpoints lack optimizer states, which are crucial for estimating weight divergence, we resume pretraining for a few more steps using Huggingface. We resume training from these checkpoints to evaluate different quantization schemes.

For the 70B model, we pretrain in BF16 precision for 10K steps and continue training up to 25K steps. Even excluding activations, training a 70B model requires approximately 1120 GB of GPU memory solely for model weights, gradients, and optimizer states [59]. Storing intermediate activations during backpropagation imposes additional memory overhead. Due to the high resource demands and lack of public intermediate checkpoints for 70B models, we limit our experiments on this model to this training window only.

Training Datasets. We adhere to the recommended guidelines for each model to download and preprocess the corresponding training datasets. For TinyLlama, we utilize a mixture of the SlimPajama and StarcoderData datasets, while for OpenLlama, we use the RedPajama [17] dataset. Since our experiments involve resuming pretraining from intermediate checkpoints rather than training models from scratch, we sample approximately 1% of the original datasets without hurting the training performance. For 70B model, we use internal industry datasets.

Quantization Format and Implementation. Since we do not have access to GPUs that natively support FP8 or FP4 formats, we implement fake quantization to emulate the quantize and dequantize operations during training. Our fake quantization function is designed to support a variety of floating-point formats less than 16 bits. Following the DeepSeek-V3 [44] FP8 training recipe, we apply 1x128 tile-wise quantization for input activations and gradients, and 128x128 block-wise quantization for weights. This configuration is chosen to balance precision and computational efficiency while minimizing quantization-induced errors. When applying the FP4 format to output gradients, we utilize stochastic rounding [19] that avoids training to stagnate by probabilistically rounding the values to the two nearest values [13, 18, 40].

Baselines To our knowledge, no prior work systematically searches for fine-grained layer-specific quantization schemes for LLM pre-training. To evaluate the effectiveness of SNIP, we compare it against the following baselines:

- **Uniform Precision:** Models trained with uniform precision across all layers using BF16, FP8 or FP4.
- **Empirical Coarse-grained Quantizations:** Higher precision is assigned to sensitive layers based on the empirical observations in prior work. This includes the first and last few layers (*E-layer-id*) [23, 85] and MLP layers in each transformer block (*E-layer-type*).
- **Fine-Grained Quantization by Error Minimization:** Layer-specific configurations optimized to minimize absolute

Table 1. Accuracy comparison of test benchmarks across different quantization schemes for TinyLlama at 50k checkpoint. We **boldface** the best performing quantization schemes with the fixed efficiency (fractions of FP4 FLOPS) from 25% to 75%. It demonstrates that SNIP consistently outperforms all the other quantization schemes, and is very close to the BF16 baseline.

Fraction of FP4 FLOPS	Quant schemes	Math		Aggregate	Commonsense					Average
		ARC_c	ARC_e	MMLU	BoolQ	HellaSwag	Obqa	PiQa	WinoGrande	
0	BF16	24.15	46.09	26.56	59.72	43.94	31.00	67.41	54.85	44.22
	FP8	23.98	45.71	26.61	60.15	44.04	31.20	67.46	54.46	44.20
25%	SNIP	24.06	45.75	26.73	60.28	43.84	31.20	67.25	54.78	44.24
	min-abs-err	24.06	45.75	26.72	60.55	43.99	30.20	67.57	53.99	44.10
	min-rel-err	24.49	45.71	26.22	60.52	43.72	30.60	67.68	54.30	44.16
	random0	23.21	39.44	25.32	59.42	35.58	28.40	63.76	51.54	40.83
	random1	23.98	29.08	25.69	39.39	28.22	23.80	54.08	49.49	34.22
	random2	24.06	44.65	26.14	61.25	43.07	30.00	67.30	53.75	43.78
	E-layer-type	27.82	26.43	22.95	37.83	26.01	25.60	48.97	50.04	33.21
	SNIP	24.06	45.41	27.04	60.43	43.79	30.40	67.57	54.54	44.16
50%	min-abs-err	25.60	26.98	25.41	38.04	26.50	25.80	51.25	50.04	33.70
	min-rel-err	27.56	28.16	24.39	37.83	26.64	26.40	52.34	51.22	34.32
	random0	26.62	27.65	23.09	37.83	27.26	24.60	51.96	49.64	33.58
	random1	28.41	27.53	24.79	39.36	27.88	26.20	51.63	49.57	34.42
	random2	24.15	44.61	25.76	61.25	42.74	30.40	66.92	53.04	43.61
	E-layer-id	26.37	26.56	24.80	37.83	27.19	25.20	49.46	48.70	33.26
	E-layer-type	26.71	26.98	22.94	37.83	26.47	24.20	50.22	49.57	33.11
	SNIP	24.40	46.04	26.68	60.76	43.46	30.80	67.41	54.14	44.21
75%	min-abs-err	27.22	26.52	24.66	37.83	26.40	25.40	50.92	49.25	33.53
	min-rel-err	25.94	26.18	24.28	37.83	26.50	25.80	51.09	50.67	33.54
	random0	27.56	27.15	25.30	37.83	26.21	26.20	51.14	49.88	33.91
	random1	28.67	27.57	24.75	37.83	25.99	24.60	51.41	50.20	33.88
	random2	27.39	27.36	25.74	37.83	26.56	26.00	50.76	49.33	33.87
	SNIP	24.23	46.00	26.83	61.07	43.16	31.80	67.36	53.67	44.27
80%	SNIP	25.60	26.64	26.50	37.83	26.55	25.80	52.99	50.20	34.01
100%	FP4	26.11	27.02	23.12	37.83	25.97	23.60	50.33	51.46	33.18

(*min-abs-err*) or relative quantization error (*min-rel-err*) for each tensor, focusing on local metrics without considering overall training quality. For a fair comparison, we also use the ILP solver as in Section 5.2 where the quality loss Q is the absolute or relative quantization error.

- Random Fine-grained Quantization schemes: Precision levels are randomly assigned to each layer (*random*) to evaluate robustness.

Efficiency Metric: Fraction of FP4 FLOPs. Lacking access to hardware that supports both FP8 and FP4 (e.g., Blackwell) for end-to-end runtime measurements, we use the fraction of FP4 FLOPs as an efficiency metric. This metric represents the proportion of FLOPs in the model’s *linear layers* executed with FP4 precision, while the remaining FLOPs are executed with FP8. To our best knowledge, the latest FP4 training studies [14, 68, 79] rely on simulation. Our goal is to guide early-stage design and estimate system-level impact ahead of broad hardware availability.

Evaluation. We evaluate the LLM performance using the training loss and LM-Evaluation-Harness framework [27], a widely adopted tool for assessing large language models (LLMs). We use training loss as one metric due to its strong correlation with quality under fixed settings. Following common practices for LLM evaluation, we cover several key categories: 1) Math and reasoning: ARC Easy (ARC-e), ARC Challenge (ARC-c) [16]; 2) Aggregate: MMLU [31]; 3) Commonsense understanding: BoolQ [15], PiQA [7], HellaSwag [82], OpenBookQA [51] and WinoGrande [61]. All

evaluations are conducted in 0-shot, with the exception of MMLU, which is evaluated using 5-shot prompting.

Software and Hardware Stack. Our experiments are conducted using the Huggingface training framework [76] with Distributed Data Parallelism (DDP) for efficient training. The experiments for 1B and 3B models are executed on a single node equipped with 4 NVIDIA A40 GPUs, each with 40GB of memory, while the experiments for 7B model are executed on a single node with 8 NVIDIA A100 GPUs, each with 80 GB of memory. For 3B and 7B models, we additionally use Deepspeed [59] Zero stage 1 that partitions the optimizer states across GPU ranks to fit the model into the GPUs. We conduct the 70B model experiments on 64 H100 GPUs using a Megatron-like [65] training framework, configured with fully sharded data parallelism (FSDP) = 2, tensor parallelism (TP) = 4, and pipeline parallelism (PP) = 8.

For the solution of ILP, we utilize the `scipy.optimize.milp` function, which is a wrapper of the HiGHS linear optimization software [30]. We set the time limit for every ILP solution to 30 seconds, where it usually takes a few seconds.

6.2 Results

6.2.1 SNIP Outperforms Other Quantization Schemes.

In this section, we use extensive experiment results on different checkpoints, different models, and different efficiency savings to demonstrate the effectiveness of the SNIP over all the other baselines.

Across Different Efficiency Savings Table 1 highlights the consistent superiority of SNIP over other quantization

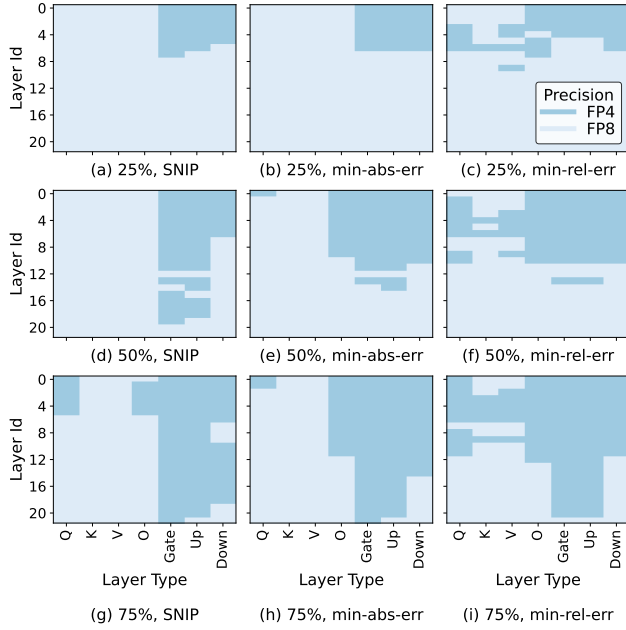


Figure 7. Per-layer precision assignments at 25%, 50%, and 75% FP4 FLOPs across different quantization schemes.

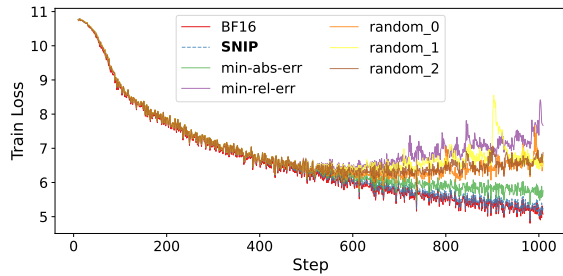


Figure 8. Training loss curve for BF16 (baseline) and different quantization schemes under a 75% FP4 FLOPs efficiency budget for TinyLlama 1B model.

schemes for the TinyLlama model at the 50k-step checkpoint, across all test benchmarks and FP4 FLOP fractions, particularly as the required efficiency savings (fractions of FP4 FLOPs) increase. For fractions of 25%, 50%, and 75% FP4 FLOPs, SNIP achieves the highest average scores, closely approximating the BF16 baseline (44.22), while all other methods experience a significant decline in performance. For instance, at 25% FP4 FLOPs, SNIP achieves an average score of 44.24, outperforming all other quantization schemes. Notably, some methods, such as *min-abs-err* and *min-rel-err*, also maintain relatively high accuracy, though they still fall short of SNIP. At 50% FP4 FLOPs, SNIP maintains its high performance with an average score of 44.16, while methods like *min-abs-err* and *min-rel-err* fail to retain competitive results. The divergence becomes more pronounced at 75%

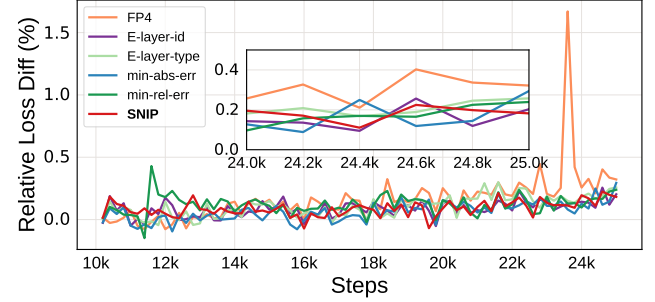


Figure 9. Relative training loss difference over BF16 (baseline) for the Llama 70B dense model from 10k to 25k steps. FP4 means all layers are using FP4 precision. And different quantization schemes are under a 50% FP4 FLOPs efficiency budget.

FP4 FLOPs, where SNIP achieves an average score of 44.21, significantly outperforming random baselines and heuristic methods, which is close to the BF16 baseline. We explain the reasons for the poor performance of other quantization schemes in the following paragraphs.

Coarse-grained quantization schemes, such as *E-layer-type* and *E-layer-id*, leverage empirical knowledge to assign FP8 precision to specific layer types (e.g., MLPs or down-projection layers) or layer indices (e.g., the first and last layers) deemed more sensitive to precision loss, while using FP4 for the remaining layers. While effective at lower efficiency budgets (e.g., 25% FP4 FLOPs) by conservatively prioritizing sensitive layers, they lack the adaptability to fully utilize the fine-grained design space of per-layer quantization. At higher efficiency budgets (e.g., 50% FP4 FLOPs), these schemes demonstrate robustness compared to other quantizations but still fall short of the superior performance achieved by SNIP, which consistently delivers better accuracy across all efficiency budgets.

Layer-wise heuristics, such as *min-abs-err* and *min-rel-err*, prioritize minimizing local quantization errors while ignoring global training dynamics. This approach works well at lower FP4 fractions (e.g., 25%), but as the FP4 fraction increases, the lack of global information leads to significant performance degradation.

Figures 7 visualize the per-layer precision assignments of different quantization schemes with different efficiency budgets. At a lower efficiency budget, like 25% FP4 FLOPs, SNIP makes layer-wise precision choices similar to those of *min-abs-err* and *min-rel-err*, resulting in comparable accuracy, as shown in Table 1. However, at 50% FP4 FLOPs, clear differences emerge; *min-abs-err* and *min-rel-err* typically assign lower precision to early layers with minimal quantization error, potentially ignoring their aggregate effect on training loss. At a 75% FP4 FLOPs efficiency, SNIP opts for higher precision in the down projection layers of middle

Table 2. Accuracy comparison across quantization schemes under a fixed efficiency budget, evaluated at different checkpoints and model sizes. BF16 serves as the baseline.

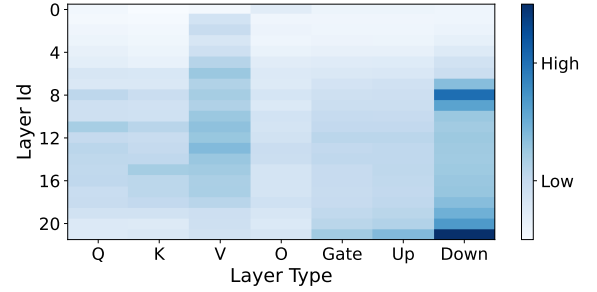
Model	TinyLlama 1B			OpenLlama 3B		OpenLlama 7B	
	5k	50k	240k	50k	100k	50k	100k
BF16	39.16	44.22	46.13	49.94	51.85	52.46	54.42
SNIP	39.15	44.21	45.78	49.72	51.72	52.43	54.45
min-abs-err	39.07	33.53	45.54	49.95	51.99	52.15	54.40
min-rel-err	33.11	33.54	45.57	50.19	51.93	52.42	54.34
random0	33.71	33.91	45.23	48.16	49.59	49.52	52.87
random1	33.24	33.88	32.86	49.16	51.04	49.05	51.84
random2	38.96	33.87	33.55	48.35	51.32	50.13	52.08

Table 3. Accuracy comparison over BF16 across different quantization schemes under a 50% efficiency budget for the Llama 70B dense model.

Model	ARC_c	MMLU	HellaSwag
FP8	-0.86	-0.01	0.24
FP4	0.17	-0.37	0.18
SNIP	-0.17	-0.12	0.59
E-layer-id	-0.52	-0.73	-0.04
E-layer-type	-0.43	0.01	-0.07
min-abs-err	0.94	-0.57	0.27
min-rel-err	-0.77	-0.45	0.09

layers, unlike *min-rel-err*, which focuses on the final layers. SNIP demonstrates a more balanced and globally optimized precision assignment compared to the other two methods. **Training Loss when Training From Scratch** Figure 8 presents the training loss over 1k steps for the TinyLlama model, trained from scratch with BF16 (baseline) and various quantization schemes under a 75% FP4 FLOPs efficiency budget, all using the same hyperparameters. The loss curves for BF16 and SNIP nearly overlap, with SNIP exhibiting a slightly higher training loss than BF16. Specifically, the training loss is 5.27 for BF16 and 5.34 for SNIP. In contrast, all other quantization schemes with the same efficiency budget fail to maintain stable training and exhibit clear divergence, underscoring the effectiveness of SNIP in preserving training stability while achieving substantial efficiency gains.

Across Different Checkpoints and Models Table 2 highlights the robustness of SNIP across different models and training phases. The efficiency budget is 75% for TinyLlama 1B model and 50% for OpenLlama 3B, 7B model because OpenLlama models are more sensitive to precision loss. It consistently delivers competitive accuracy for TinyLlama 1B and OpenLlama 3B and 7B, demonstrating scalability to larger models. Additionally, SNIP maintains stable performance across various training checkpoints, ensuring reliability throughout different training stages. While in OpenLlama 3B, SNIP shows slightly lower test accuracy than *min-abs-err* and *min-rel-err*, its training loss remains consistently lower. The fact that *min-abs-err* and *min-rel-err* outperform the BF16 baseline in test accuracy suggests potential noise in their evaluations rather than true improvements.

**Figure 10.** Heatmap of the layer-wise quality loss to FP4 quantization for 1B model at 50k step checkpoint. The darker regions indicate higher sensitivity to FP4 precision loss.

We further evaluate the training stability of quantization on the 70B dense model by measuring the relative training loss difference with respect to BF16 precision from 10k to 25k steps (Figure 9). A lower value indicates better stability. When all layers are trained with FP4 precision, the model exhibits a gradual divergence, though notably slower than that observed in smaller models such as TinyLlama-1B (Figure 8). This observation is consistent with prior results on quantization for LLM inference [11, 25, 42, 48], which show that larger models are more resilient to precision loss. Figure 9 further shows that, under a 50% FP4 FLOPs efficiency budget, our SNIP scheme achieves loss curves that remain closely aligned with BF16, while heuristic schemes, such as *min-rel-err* and *E-layer-type* (assigning FP8 to the gate and up-projection layers of the MLP while using FP4 for the remaining layers), exhibit occasional spikes and larger deviations. It can be seen from the figure that SNIP and *E-layer-id* (assigning FP4 to the middle 50% of layers and FP8 to the rest) outperforms other quantization schemes.

Table 3 reports accuracy differences over BF16 on evaluation datasets. "+" means improvement and "-" means degradation. FP4 baselines suffer from accuracy drops in MMLU, while heuristic selection schemes produce inconsistent results across tasks. In contrast, SNIP consistently delivers stable accuracy, and in some cases even surpasses FP4 and FP8 baselines. Taken together, these results show that SNIP not only provides smoother training dynamics but also achieves superior downstream accuracy compared to heuristic quantization strategies across models from 1B, 3B, 7B to 70B.

6.3 In-Depth Analysis

Quality Importance of Each Layer. To better understand the impact of quantizing each layer in FP4 format, we visualize the "importance" of each layer in Figure 10, which corresponds to quality loss (Q) in Section 5. The figure highlights key observations: the last layer's MLP stands out as the most critical, requiring higher precision to maintain model quality. Similarly, down-projection (Down) layers in the MLP structure, particularly in the later layers, show higher sensitivity,

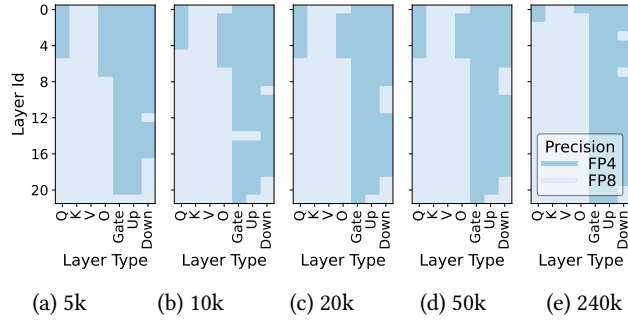


Figure 11. Evolution of per-layer precision assignments determined by SNIP at 75% FP4 FLOPs across different training checkpoints (5k, 10k, 20k, 50k, 240k) for TinyLlama model.

emphasizing their importance in preserving transformed information. Among the attention layers (Q, K, V), the V (Value) layers stand out as more sensitive than Q (Query) and K (Key) layers, despite operating on the same input activations, emphasizing their importance in the attention mechanism.

These findings underscore the need for fine-grained, layer-specific quantization strategies, as demonstrated by SNIP, to effectively balance quality and efficiency while providing insights into model behavior.

Generate Quantization Scheme Periodically. Figure 11 demonstrates how SNIP adjusts per-layer precision assignments across training checkpoints, reflecting the evolving importance of layers. From 5k to 50k steps, precision assignments remain stable, suggesting little change in layer sensitivities in a short time period. At 240k steps, clear differences emerge, with more layers assigned higher precision (FP8). Notably, the first few layers gain importance in later stages, receiving higher precision, while the last few layers shift to lower precision (FP4). This highlights SNIP’s ability to adapt precision dynamically, balancing efficiency and model quality as training progresses. Based on these findings, we recommend generating updated quantization schemes periodically, approximately every 100k steps. SNIP introduces runtime overheads due to the collection of statistics and processing. Steps 1 to 3 involve three additional training iterations, each incurs an overhead of roughly 2–3 times that of a normal training iteration, approximately 10 minutes. Steps 4 and 5, which handle optimization, are offloaded to the CPU, allowing GPU training to continue unaffected and take about 15 minutes. This setup ensures minimal disruption to the training process.

Incorporating Pipeline Parallelism. Figure 12 illustrates the timeline of pipeline parallelism for the TinyLlama model using SNIP’s fine-grained mixed precision training with 50% efficiency savings and 4 pipeline stages for forward and backward pass. The TinyLlama model consists of 22 layers, which

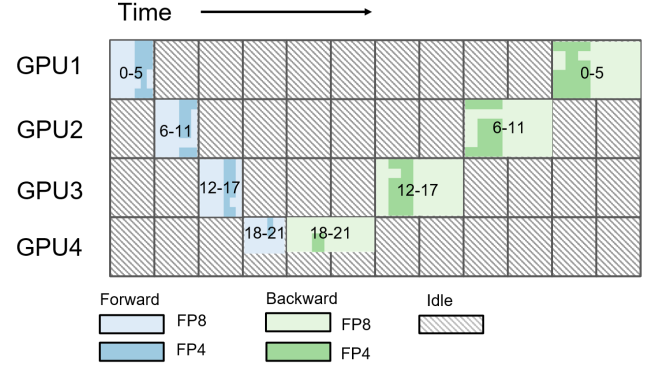


Figure 12. Timeline of pipeline parallelism of TinyLlama model using SNIP with 50% efficiency savings with four stages. Each stage processes specific layers, marked by their layer id, and layer-specific precisions are visualized in 2D heatmaps.

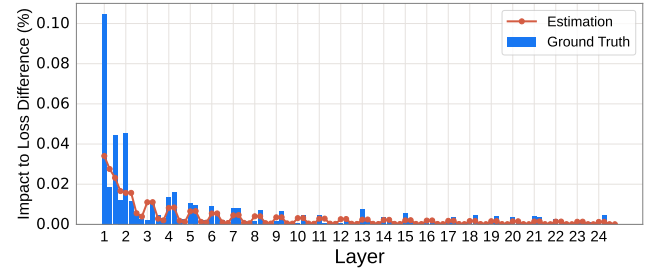


Figure 13. SNIP-estimated vs. ground-truth per-layer loss impact when quantizing weights and activations, showing close alignment across all layers.

are evenly distributed across the first 3 stages, each processing 6 layers, while the 4th stage processes the final four layers. Layer-specific precisions, determined by SNIP, are visualized in 2D heatmaps for each stage, where the x-axis represents layer types, and the y-axis represents layer IDs. Note that although we have less than 50% FP4 FLOPs in the 4th stage, since it processes 4 layers rather than 6 layers in the other 3 stages, the overall efficiency remains balanced across the pipeline. The efficient distribution of layers and mixed precision assignments helps minimize idle time and maximize overall throughput in pipeline parallelism.

The Estimated vs. Actual Error Propagation. To ensure the reliability of our estimation method before integrating it into the ILP formulation, we validated the estimated error propagation against actual measured errors. In the experiments, we quantize each layer individually and perform a forward pass to measure the loss difference relative to the BF16 baseline, obtaining the ground-truth impact. We then apply the expression described in Section 4.2 to estimate the loss impact when quantizing the weights and activations of different layers. As shown in Figure 13, the estimated

per-layer impact on forward-pass loss closely matches the ground-truth measurements across all layers, capturing both the relative magnitudes and overall trends. This strong agreement demonstrates that our estimation provides an accurate proxy for actual error propagation, enabling effective layer selection in the subsequent ILP optimization.

Memory Overhead of SNIP. SNIP collects statistics during the forward and backward passes to estimate loss and weight divergence. To improve sensitivity estimation, we replace global Frobenius norms with a row-wise formulation, which stores only M or N additional values for an $M \times N$ tensor. This overhead is negligible relative to tensor size, and in practice the GPU memory overhead of SNIP is under 1%.

7 Related Work

Mixed Precision Training for LLM Pretraining. Mixed precision training has emerged as a technique to balance computational efficiency and model accuracy in the training of large-scale deep learning models. It was first introduced in [49], which demonstrated the feasibility of training deep neural networks using FP16 for most operations, while maintaining FP32 master copies of weights to ensure numerical stability during updates. Subsequent advancements explored lower-precision formats for GEMM operations, such as FP8 [44, 57, 58], FP4 [73], MX format [60, 68], and INT8 [77]. Frameworks like Pytorch’s AMP (Automatic Mixed Precision) [4] and NVIDIA’s Apex [53] automated the use of mixed precision training. Despite these advances, most mixed precision training frameworks use a uniform precision policy across all layers, overlooking potential efficiency gains from layer-wise precision tuning. Recent work has begun to incorporate layer-wise information into training precision policies. FGMP [32] leverages Fisher information to guide mixed-precision quantization, ACCORDION [3] uses Hessian-based analysis to detect critical regimes, and Egeria [74] introduces a plasticity metric to measure divergence in intermediate activations between a training model and its reference counterpart. To the best of our knowledge, SNIP is the first to enable sensitivity-aware precision selection for LLM pretraining by jointly considering both loss divergence and weight divergence, and to demonstrate its effectiveness at scale from 1B to 70B models.

Adaptive Mixed Precision for LLM Inference. Beyond uniform-precision training, several works have explored adaptive or layer-wise mixed precision for LLM inference. Methods like LLM-MQ [41], Slim-LLM [35], MixLLM [86], AptQ [29], and RaanA [80] select precision by minimizing local layer quantization error or consider the impact on loss in the forward pass only, but neglect the cumulative impact of quantization on overall training loss and weight updates. In contrast, SNIP targets pretraining, where weight dynamics are stronger and higher precision is often required for stability.

FP4 LLM Training and Inference. Building on the success of FP8 pretraining [44], recent studies have begun exploring FP4 [1] as the next frontier for reducing training cost while maintaining stability and accuracy, enabled by NVIDIA Blackwell’s native FP4 support. Prior studies examine quantization recipes such as block size and rounding [14, 68, 79], as well as algorithmic enhancements including random Hadamard transforms (RHT) [68] and differentiable gradient estimators (DGE) [73] to improve FP4 training accuracy. Despite these advances, a noticeable gap remains between native FP4 training and high-precision baselines (e.g., BF16), suggesting the need for finer-grained mixed-precision approaches. However, FP4 training still lags behind high-precision baselines (e.g., BF16), motivating finer-grained mixed-precision approaches. Complementary work on FP4 inference, such as attention smoothing [83] and outlier handling [38], is orthogonal and can be combined with SNIP for further gains.

Quantization for LLM Inference and Finetuning. Quantization has been widely explored for LLM inference to reduce memory and computation costs, leveraging finer quantization granularity, specialized data formats, and outlier suppression techniques. These efforts primarily fall into two categories: post-training quantization (PTQ) [20, 34, 37, 43, 47, 62, 71, 75, 78, 81], which applies quantization to pretrained models without retraining, and quantization-aware training (QAT) [10, 45, 64], which retrains models to adapt to quantized inference. Additionally, parameter-efficient fine-tuning (PEFT) techniques such as LoRA [33] have been combined with quantization in fine-tuning approaches like QLoRA [21]. These inference-stage quantization techniques do not directly apply to LLM pretraining because they have substantial overhead and do not consider the overall training quality. Orthogonal to these, various works propose new quantization formats (e.g., LUQ [13]), whereas SNIP selects among existing formats (e.g., FP4/FP8) to balance training efficiency and accuracy, and remains compatible with emerging ones.

8 Conclusion

We propose SNIP, a layerwise mixed-precision framework that introduces a new level of granularity in precision selection per linear layer. By quantifying training quality loss through loss divergence and weight divergence, we formulate per-layer quantization as an Integer Linear Programming (ILP) problem to achieve optimal precision assignments. SNIP balances training quality and efficiency while seamlessly integrating into LLM training pipelines. Experiments across 1B, 3B, 7B, and 70B models and different training phases demonstrate that SNIP outperforms other heuristic-based approaches, achieving up to 80% FLOPs efficiency savings in FP4 while maintaining accuracy close to the BF16 baseline.

References

- [1] Felix Abecassis, Anjolie Agrusa, Dong Ahn, Jonah Alben, Stefania Alborghetti, Michael Andersch, Sivakumar Arayandi, Alexis Bjorlin, Aaron Blakeman, Evan Briones, et al. 2025. Pretraining large language models with nvfp4. *arXiv preprint arXiv:2509.25149* (2025).
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Saurabh Agarwal, Hongyi Wang, Kangwook Lee, Shivaram Venkataraman, and Dimitris Papailiopoulos. 2021. Adaptive gradient communication via critical learning regime identification. *Proceedings of Machine Learning and Systems* 3 (2021), 55–80.
- [4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.
- [5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [6] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013).
- [7] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 7432–7439.
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [9] Adam Casson. 2023. Transformer flops. <https://www.adamcasson.com/posts/transformer-flops>.
- [10] Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, and Ping Luo. 2024. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062* (2024).
- [11] Mengzhao Chen, Chaoyi Zhang, Jing Liu, Yutao Zeng, Zeyue Xue, Zhiheng Liu, Yunshui Li, Jin Ma, Jie Huang, Xun Zhou, et al. 2025. Scaling Law for Quantization-Aware Training. *arXiv preprint arXiv:2505.14302* (2025).
- [12] Yuzong Chen, Jian Meng, Jae-sun Seo, and Mohamed S Abdelfattah. 2024. BBS: Bi-directional Bit-level Sparsity for Deep Learning Acceleration. *arXiv preprint arXiv:2409.05227* (2024).
- [13] Brian Chmiel, Ron Banner, Elad Hoffer, Hilla Ben-Yaacov, and Daniel Soudry. 2023. Accurate neural training with 4-bit matrix multiplications at standard formats. In *The Eleventh International Conference on Learning Representations*.
- [14] Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. 2025. FP4 All the Way: Fully Quantized Training of LLMs. *arXiv preprint arXiv:2505.19115* (2025).
- [15] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044* (2019).
- [16] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [17] Together Computer. 2023. *RedPajama-Data: An Open Source Recipe to Reproduce LLaMA training dataset*. <https://github.com/togethercomputer/RedPajama-Data>
- [18] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv preprint arXiv:1602.02830* (2016).
- [19] Matteo Croci, Massimiliano Fasi, Nicholas J Higham, Theo Mary, and Mantas Mikaitis. 2022. Stochastic rounding: implementation, error analysis and applications. *Royal Society Open Science* 9, 3 (2022), 211631.
- [20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8() 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. 30318–30332.
- [21] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).
- [22] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [23] Razvan-Gabriel Dumitru, Vikas Yadav, Rishabh Maheshwary, Paul-Ioan Clotan, Sathwik Tejaswi Madhusudhan, and Mihai Surdeanu. 2024. Layer-Wise Quantization: A Pragmatic and Effective Method for Quantizing LLMs Beyond Integer Bit-Levels. *arXiv preprint arXiv:2406.17415* (2024).
- [24] Elias Frantar and Dan Alistarh. 2022. Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems* 35 (2022), 4475–4488.
- [25] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [26] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [27] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. A framework for few-shot language model evaluation. [doi:10.5281/zenodo.12608602](https://doi.org/10.5281/zenodo.12608602)
- [28] Xinyang Geng and Hao Liu. 2023. *OpenLLaMA: An Open Reproduction of LLaMA*. https://github.com/openlm-research/open_llama
- [29] Ziyi Guan, Hantao Huang, Yupeng Su, Hong Huang, Ngai Wong, and Hao Yu. 2024. Aptq: Attention-aware post-training mixed-precision quantization for large language models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [30] J Hall, I Galabova, L Gottwald, and M Feldmeier. 2023. HiGHS—high performance software for linear optimization.
- [31] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [32] Coleman Hooper, Charbel Sakr, Ben Keller, Rangharajan Venkatesan, Kurt Keutzer, Sophia Shao, and Bruce Khailany. 2025. FGMP: Fine-Grained Mixed-Precision Weight and Activation Quantization for Hardware-Accelerated LLM Inference. *arXiv preprint arXiv:2504.14152* (2025).
- [33] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*

- (2021).
- [34] Wei Huang, Yangdong Liu, Haotong Qin, Ying Li, Shiming Zhang, Xianglong Liu, Michele Magno, and Xiaojuan Qi. 2024. Billm: Pushing the limit of post-training quantization for llms. *arXiv preprint arXiv:2402.04291* (2024).
 - [35] Wei Huang, Haotong Qin, Yangdong Liu, Yawei Li, Qinshuo Liu, Xianglong Liu, Luca Benini, Michele Magno, Shiming Zhang, and Xiaojuan Qi. 2024. SliM-LLM: Saliency-driven mixed-precision quantization for large language models. *arXiv preprint arXiv:2405.14917* (2024).
 - [36] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. 2019. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322* (2019).
 - [37] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
 - [38] Janghwan Lee, Jiwoong Park, Jinseok Kim, Yongjik Kim, Jungju Oh, Jinwook Oh, and Jungwook Choi. 2025. Amxfp4: Taming activation outliers with asymmetric microscaling floating-point for 4-bit llm inference. In *Findings of the Association for Computational Linguistics: ACL 2025*. 14993–15013.
 - [39] Jun Haeng Lee, Tobi Delbruck, and Michael Pfeiffer. 2016. Training deep spiking neural networks using backpropagation. *Frontiers in neuroscience* 10 (2016), 508.
 - [40] Hao Li, Soham De, Zheng Xu, Christoph Studer, Hanan Samet, and Tom Goldstein. 2017. Training quantized nets: A deeper understanding. *Advances in Neural Information Processing Systems* 30 (2017).
 - [41] Shiyao Li, Xuefei Ning, Ke Hong, Tengxuan Liu, Luning Wang, Xiuhong Li, Kai Zhong, Guohao Dai, Huazhong Yang, and Yu Wang. 2023. Llm-mq: Mixed-precision quantization for efficient llm deployment. In *The Efficient Natural Language and Speech Processing Workshop with NeurIPS*, Vol. 9. 3.
 - [42] Xinlin Li, Osama Hanna, Christina Fragouli, and Suhas Diggavi. 2025. ICQuant: Index Coding enables Low-bit LLM Quantization. *arXiv preprint arXiv:2505.00850* (2025).
 - [43] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
 - [44] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
 - [45] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888* (2023).
 - [46] I Loshchilov. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
 - [47] Yuexiao Ma, Huixia Li, Xiawu Zheng, Feng Ling, Xuefeng Xiao, Rui Wang, Shilei Wen, Fei Chao, and Rongrong Ji. 2024. Affinequant: Affine transformation quantization for large language models. *arXiv preprint arXiv:2403.12544* (2024).
 - [48] Vladimir Malinovskii, Denis Mazur, Ivan Ilin, Denis Kuznedelev, Konstantin Burlachenko, Kai Yi, Dan Alistarh, and Peter Richtarik. 2024. Pv-tuning: Beyond straight-through estimation for extreme llm compression. *Advances in Neural Information Processing Systems* 37 (2024), 5074–5121.
 - [49] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2018. Mixed Precision Training. In *International Conference on Learning Representations*.
 - [50] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. 2022. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433* (2022).
 - [51] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789* (2018).
 - [52] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
 - [53] NVIDIA. 2019. APEX. <https://github.com/nvidia/apex>.
 - [54] Nvidia. 2022. NVIDIA H100 Tensor Core GPU Architecture. <https://resources.nvidia.com/en-us-tensor-core>.
 - [55] Nvidia. 2024. Nvidia blackwell architecture technical brief. <https://resources.nvidia.com/en-us-blackwell-architecture>.
 - [56] Yunjie Pan, Jiecao Yu, Andrew Lukefahr, Reetuparna Das, and Scott Mahlke. 2023. BitSET: Bit-serial early termination for computation reduction in convolutional neural networks. *ACM Transactions on Embedded Computing Systems* 22, 5s (2023), 1–24.
 - [57] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, et al. 2023. Fp8-llm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313* (2023).
 - [58] Sergio P Perez, Yan Zhang, James Briggs, Charlie Blake, Josh Levy-Kramer, Paul Balanca, Carlo Luschi, Stephen Barlow, and Andrew William Fitzgibbon. 2023. Training and inference of large language models using 8-bit floating point. *arXiv preprint arXiv:2309.17224* (2023).
 - [59] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
 - [60] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
 - [61] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
 - [62] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137* (2023).
 - [63] Haihao Shen, Naveen Mellempudi, Xin He, Qun Gao, Chang Wang, and Mengni Wang. 2024. Efficient post-training quantization with fp8 formats. *Proceedings of Machine Learning and Systems* 6 (2024), 483–498.
 - [64] Xuan Shen, Zhenglun Kong, Changdi Yang, Zhaoyang Han, Lei Lu, Peiyan Dong, Cheng Lyu, Chih-hsiang Li, Xuehang Guo, Zhihao Shu, et al. 2024. EdgeQAT: Entropy and Distribution Guided Quantization-Aware Training for the Acceleration of Lightweight LLMs on the Edge. *arXiv preprint arXiv:2402.10787* (2024).
 - [65] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
 - [66] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multi-modal models. *arXiv preprint arXiv:2312.11805* (2023).

- [67] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [68] Albert Tseng, Tao Yu, and Youngsuk Park. 2025. Training LLMs with MXFP4. *arXiv preprint arXiv:2502.20586* (2025).
- [69] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [70] Roman Vershynin. 2018. *High-dimensional probability: An introduction with applications in data science*. Vol. 47. Cambridge university press.
- [71] Changyuan Wang, Ziwei Wang, Xiuwei Xu, Yansong Tang, Jie Zhou, and Jiwen Lu. 2024. Q-VLM: Post-training Quantization for Large Vision-Language Models. *arXiv preprint arXiv:2410.08119* (2024).
- [72] Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. 2019. Haq: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8612–8620.
- [73] Ruizhe Wang, Yeyun Gong, Xiao Liu, Guoshuai Zhao, Ziyue Yang, Baining Guo, Zhengjun Zha, and Peng Cheng. 2025. Optimizing Large Language Model Training Using FP4 Quantization. *arXiv preprint arXiv:2501.17116* (2025).
- [74] Yiding Wang, Decang Sun, Kai Chen, Fan Lai, and Mosharaf Chowdhury. 2023. Egeria: Efficient dnn training with knowledge-guided layer freezing. In *Proceedings of the eighteenth European conference on computer systems*. 851–866.
- [75] Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo Zhang, Ruihao Gong, Jinyang Guo, and Xianglong Liu. 2023. Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145* (2023).
- [76] T Wolf. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
- [77] Haocheng Xi, Yuxiang Chen, Kang Zhao, Kaijun Zheng, Jianfei Chen, and Jun Zhu. 2024. Jetfire: Efficient and Accurate Transformer Pre-training with INT8 Data Flow and Per-Block Quantization. *arXiv preprint arXiv:2403.12422* (2024).
- [78] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [79] Hanmei Yang, Summer Deng, Amit Nagpal, Maxim Naumov, Mohammad Janani, Tongping Liu, and Hui Guan. 2025. An Empirical Study of Microscaling Formats for Low-Precision LLM Training. In *2025 IEEE 32nd Symposium on Computer Arithmetic (ARITH)*. IEEE Computer Society, 1–8.
- [80] Yongyi Yang, Jianyang Gao, and Wei Hu. 2025. Raana: A fast, flexible, and data-efficient post-training quantization algorithm. *arXiv preprint arXiv:2504.03717* (2025).
- [81] Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xinggang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. 2023. Rptq: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089* (2023).
- [82] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- [83] Jintao Zhang, Jia Wei, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Haoxu Wang, Kai Jiang, Jun Zhu, and Jianfei Chen. 2025. Sageattention3: Microscaling fp4 attention for inference and an exploration of 8-bit training. *arXiv preprint arXiv:2505.11594* (2025).
- [84] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. *arXiv:2401.02385* [cs.CL]
- [85] Yang Zhang, Yanfei Dong, and Kenji Kawaguchi. 2024. Investigating layer importance in large language models. *arXiv preprint arXiv:2409.14381* (2024).
- [86] Zhen Zheng, Xiaonan Song, and Chuanjie Liu. 2024. Mixllm: Llm quantization with global mixed-precision between output-features and highly-efficient system design. *arXiv preprint arXiv:2412.14590* (2024).

A Proof of the Theoretical Results

A.1 Proof of Theorem 4.1

Throughout this proof we use “absolute constant” to indicate a constant value that does not depend on any variables used in the theorem.

Notice that $\mathbb{E}\|\delta\|^2 = \epsilon^2$. Let event $\mathcal{E} = \{C^{-1}\epsilon \leq \|\delta\| \leq C\epsilon\}$, where $C > 1$ is an absolute constant. From the known concentration property of Gaussian random vectors, there exists a C such that $\mathbb{P}(\mathcal{E}) \geq 0.995$. In the following we condition on all of our statements on $\mathcal{E}$.

Let $G = \nabla_x g(x) \in \mathbb{R}^{m \times d}$. Since g is smooth, a small perturbation can be approximated by the first order Taylor expansion. Specifically, we have

$$\begin{aligned} \|g(x + \delta) - g(x)\|_F &= \|G\delta + O(S\|\delta\|^2)\|_F \\ &\leq \|G\delta\|_F + O(S\epsilon^2). \end{aligned}$$

Since we assumed $S \ll \epsilon^{-2}$, the second term is negligible, and thus we focus on the first term.

Let the SVD of G be $G = U\Sigma V$, where $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{d \times d}$ be orthonormal matrices, and $\Sigma \in \mathbb{R}^{m \times d}$ be a concatenation of a $\min(d, m)$ -diagonal matrix and an all-zero matrix. We have

$$\|G\delta\|_F = \|U\Sigma V\delta\|_F = \|\Sigma\bar{\delta}\|_F,$$

where $\bar{\delta} = V\delta$ which is known to have the same distribution as δ . Let $\sigma = \{\Sigma_{i,i}\}_{i=1}^d$ be the diagonal entries of Σ (if $m < d$, we pad σ with 0 to make it a d -dimensional vector). We have

$$\|\Sigma\bar{\delta}\|_F = \sqrt{\sum_{i=1}^d \sigma_i^2 \bar{\delta}_i^2}.$$

Since $\bar{\delta}$ is identically distributed with δ , we have $\mathbb{E}\bar{\delta}_i^2 = \frac{\epsilon^2}{d}$. Moreover, since orthonormal transformations does not change the Frobenius norm, we have $\|\Sigma\|_F = \|G\|_F$. Therefore

$$\mathbb{E}\|\Sigma\bar{\delta}\|_F^2 = \sum_{i=1}^d \sigma_i^2 \mathbb{E}\bar{\delta}_i^2 = \frac{\epsilon^2}{d} \|\Sigma\|_F^2 = \frac{\epsilon^2}{d} \|G\|_F^2.$$

From Bernstein’s inequality [70], we know the value of $\|\Sigma\bar{\delta}\|_F^2$ is concentrated around its expectation. Specifically,

there exists an absolute constant C_2 such that

$$\begin{aligned} \mathbb{P}\left\{\left|\|\Sigma\bar{\delta}\|_F^2 - \frac{\epsilon^2}{d} \|G\|_F^2\right| > t\right\} &\leq 2 \exp\left(-\frac{C_2 t^2}{\frac{\epsilon^4}{d^2} \sum_{i=1}^d \sigma_i^4 + t \cdot \frac{\epsilon^2}{d} \max_{i=1}^d \sigma_i^2}\right) \\ &\leq 2 \exp\left(-\frac{C_2 t^2}{\frac{\epsilon^4}{d^2} \|G\|_F^4 + t \cdot \frac{\epsilon^2}{d} \|G\|_F^2}\right). \end{aligned}$$

It is easy to see that there exists an absolute constant C_3 , such that with probability at least 0.995, we have

$$\|G\delta\|_F = \|\Sigma\bar{\delta}\|_F \leq C_3 \cdot \frac{\epsilon}{\sqrt{d}} \|G\|_F. \quad (6)$$

Since our statement is conditioned on $\mathcal{E}$, the probability of the overall statement is at least

$$1 - \mathbb{P}(\mathcal{E}^c) - \mathbb{P}\left\{\|G\delta\|_F > C_3 \cdot \frac{\epsilon}{\sqrt{d}} \|G\|_F\right\} \geq 0.99.$$

A.2 Proof of Theorem 4.2

WLOG we only need to prove the theorem for $m = 1$, since in the case of $m > 1$ we only need to apply the 1-d conclusion to each output entry.

Let $Y \sim \mathcal{N}(0, I_d)$ be a d -dimensional standard Gaussian vector. We have δ_ϵ is identically distributed with ϵY . Using the dominated convergence theorem and the fact that continuous operations commute with limit, we only need to prove the eq.(1) with the right-hand side being replaced with

$$\mathbb{E}\left|\lim_{\epsilon \rightarrow 0} \frac{g(x) - g(x + \epsilon Y)}{\epsilon}\right|^2.$$

Let $G \in \mathbb{R}^d$ be the gradient of g at point x . Notice that $D_Y(x) = \lim_{\epsilon \rightarrow 0} \frac{g(x) - g(x + \epsilon Y)}{\epsilon}$ is the directional derivative of g at point x in the direction of Y . Since g is smooth, we have $D_Y(x) = \langle G, Y \rangle$. Thus we have

$$\begin{aligned} \mathbb{E}\left|\lim_{\epsilon \rightarrow 0} \frac{g(x) - g(x + \epsilon Y)}{\epsilon}\right|^2 &= \mathbb{E}\langle G, Y \rangle^2 \\ &= \sum_{i=1}^d G_i^2 \mathbb{E}Y_i^2 \\ &= \sum_{i=1}^d G_i^2 \\ &= \|G\|^2. \end{aligned}$$